Relatório do Projeto 2: RSA

Ana Paula Gomes de Matos (160023629), André Milhomem Araújo de Godoi (202033427),
Geilson dos Santos Sa (231006239)

***Resumo*—Este relatório apresenta projeto de desenvolvimento de programa que utiliza o sistema de criptografia Rivest–Shamir–Adleman (RSA) combinado com os esquemas Optimal Asymetric Encryption Padding (OAEP) e Probabilistic Signature Scheme (PSS) para cifrar e decifrar mensagens e para gerar e verificar assinaturas digitais.**

***Palavras-chave*—Rivest–Shamir–Adleman (RSA), Optimal Asymetric Encryption Padding (OAEP), Probabilistic Signature Scheme (PSS).**

I. Introdução

O sistema RSA, descrito por Ron Rivest, Adi Shamir e Leonard Adleman em 1977, consiste em um esquema de criptografia assimétrico que se baseia em um par de chaves (uma pública; outra privada) e combina propriedades de aritmética modular e aritmética de números primos para garantir confidencialidade, integridade e autenticidade de mensagens [1].

Na atualidade, como camada adicional de segurança, o sistema RSA é complementado pelos esquemas Optimal Asymetric Encryption Padding (OAEP), para cifragem e decifragem de mensagens, e Probabilistic Signature Scheme (PSS), para geração e verificação de assinatura digital [1].

Neste projeto, desenvolvemos um programa que implementa, com adaptações, os algoritmos RSAES-OAEP e RSASSA-PSS, descritos na RFC 8017 [2], para cifrar e decifrar mensagens e para gerar e verificar assinaturas digitais, respectivamente. A fundamentação teórica, o ambiente experimental e os resultados obtidos são descritos a seguir.

II. Fundamentação teórica

A. RSA

O sistema RSA se baseia na utilização de uma chave pública e uma chave privada, obtidas conforme a especificação abaixo.

Seja $\phi(x)$, com $x \in \mathbb{Z}^+$, o número de inteiros positivos menores que x e coprimos de x (função totiente de Euler). Podemos provar que $\phi(x) = x\prod_{i=1}^{k}(1-\frac{1}{y_i})$, sendo $y_1, \ldots, y_k$ os k distintos números primos que dividem x [3].

Sejam p e q números primos quaisquer distintos e $n = p.q$. Temos que $\phi(n) = n[(1-\frac{1}{p})(1-\frac{1}{q})] = n[(\frac{p-1}{p})(\frac{q-1}{q})] = n[\frac{(p-1)(q-1)}{pq}] = n[\frac{(p-1)(q-1)}{n}] = (p-1)(q-1)$.

Seja e um número inteiro positivo coprimo de $\phi(n)$, isto é, $mdc(e, \phi(n)) = 1$. Pelo Teorema de Bézout, sabemos que e e $\phi(n)$ são coprimos se, e somente se, e é inversível módulo $\phi(n)$, isto é, que $mdc(e, \phi(n)) = 1 \iff \exists e^{-1} \in Z^+ : ee^{-1} \equiv 1 \pmod{\phi(n)}$. Nesse contexto, seja $d \in \mathbb{Z}^+$ o inverso de e módulo $\phi(n)$, isto é, $ed \equiv 1 \pmod{\phi(n)}$ ou ainda $ed = k\phi(n) + 1, k \in Z^+$.

Sejam m um número inteiro positivo que represente uma mensagem e c um número inteiro positivo que represente a cifra dessa mensagem m.

No sistema RSA, a operação de cifragem é assim definida: $c = m^e \mod n$. Já a operação de decifragem é realizada da seguinte maneira: $m = c^d \mod n$.

Para que o sistema RSA funcione, precisamos garantir a inversibilidade das operações, isto é, provar que, ao decifrar um texto anteriormente cifrado, podemos obter o texto original.

Pelo Teorema de Euler, sabemos que $mdc(m, n) = 1 \implies m^{\phi(n)} \equiv 1 \pmod{n}$. Além disso, sabemos que $m \mod n = m, \forall m < n$.

Supondo que m e n são coprimos e que $m < n$, observamos o seguinte:

$$
\begin{aligned}
c^d \mod n &= \\
(m^e \mod n)^d \mod n &= \\
m^{ed} \mod n &= \\
m^{k\phi(n)+1} \mod n &= \\
m^{k\phi(n)}m \mod n &= \\
(m^{k\phi(n)} \mod n)(m \mod n) \mod n &= \\
((m^{\phi(n)})^k \mod n)(m \mod n) \mod n &= \\
((m^{\phi(n)} \mod n)^k \mod n)(m \mod n) \mod n &= \\
(1^k \mod n)(m \mod n) \mod n &= \\
(1 \mod n)(m \mod n) \mod n &= \\
1(m \mod n) \mod n &= \\
(m \mod n) \mod n &= \\
m \mod n &= \\
m &
\end{aligned}
$$

Podemos, aliás, provar que esse resultado ($c^d \mod n = m$) vale mesmo na remota hipótese em que m e n não sejam

coprimos[1].

Notamos que a suposição de que $m < n$ é primordial para garantir a injetividade da operação de cifragem, isto é, para assegurar que para cada mensagem m existe uma única mensagem cifrada c correspondente.

Adicionalmente, observamos que, como $ed = de, \forall e, d \in Z^+$, se utilizarmos o expoente d na operação inicial, podemos utilizar o expoente e para invertê-la. Com base nessa propriedade, podemos definir a operação de assinatura como $c = a^d \mod n$ e a operação de verificação de assinatura como $a = c^e \mod n$, sendo a o valor de uma função de *hash* aplicada sobre a mensagem m.

No sistema RSA, o par *(n,e)* é denominado chave pública e o par *(n,d)*, chave privada. Nas operações de cifragem e decifragem de uma mensagem, utilizam-se, nessa ordem, as chaves pública e privada do destinatário. Já nas operações de assinatura e verificação de assinatura, utilizam-se, nessa ordem, as chaves privada e pública do remetente.

Em razão da injetividade da operação de cifragem, o sistema RSA assume caráter determinístico: dada uma mensagem m e dadas as chaves pública (n, e) e privada (n, d), o resultado c da operação de cifragem será sempre o mesmo.

Essa característica representa uma vulnerabilidade do sistema RSA e ensejou a criação de esquemas de *padding*, como o OAEP e o PSS (que veremos a seguir), com o objetivo de adicionar uma camada de aleatoriedade ao sistema.

B. OAEP

O esquema OAEP consiste em uma série de transformações que atuam sobre a mensagem *m* previamente à operação de cifragem pelo sistema RSA. A Figura 1, extraída de [2], sintetiza esse processo.

O processo consiste dos seguintes passos:

1) Sobre uma *label* pré-determinada, aplica-se uma função de *hash*, obtendo-se a sequência $lHash$ com $hLen$ octetos (*bytes*).
2) Dado o componente n (com k octetos) das chaves do destinatário, limitamos o tamanho $mLen$ da mensagem m a ser codificada a, no máximo, $k-2hLen-2$ octetos.

[1]Mesmo que m e n não sejam coprimos, podemos provar que $c^d \mod n = m$ ou, mais especificamente, que $m^{ed} \mod n = m$. Como $n = pq$ e p e q são primos, a única hipótese em que m e n não são primos é a situação em que m é múltiplo de p ou de q. Suponhamos que m seja múltiplo de p ($m = kp, k \in \mathbb{Z}^+$). Como $m < n = pq$, nesse cenário, m é necessariamente coprimo de q. Bifurquemos a prova em dois ramos: a) $m^{ed} \equiv m \pmod p$; e b) $m^{ed} \equiv m \pmod q$. O item a pode ser provado assim: $m^{ed} \mod p = (m \mod p)^{ed} \mod p = (kp \mod p)^{ed} \mod p = 0^{ed} \mod p = 0 \mod p = 0 = kp \mod p = m \mod p$. Quanto ao item b, sabemos, pelo Teorema de Euler, que $mdc(m, q) = 1 \implies m^{\phi(q)} \equiv 1 \pmod q$ e, como q é primo, que $\phi(q) = q - 1$. Assim, $m^{\phi(q)} = m^{q-1} \equiv 1 \pmod q$. Sabemos ainda que $\phi(n) = (p-1)(q-1)$. Com base nessas premissas, observamos o seguinte: $m^{ed} \mod q = m^{j\phi(n)+1} \mod q = m^{j\phi(n)}m \mod q = (m^{j\phi(n)} \mod q)(m \mod q) \mod q = (m^{j(p-1)(q-1)} \mod q)(m \mod q) \mod q = ((m^{(q-1)})^{j(p-1)} \mod q)(m \mod q) \mod q = (((m^{(q-1)}) \mod q)^{j(p-1)} \mod q)(m \mod q) \mod q = ((1 \mod q)^{j(p-1)} \mod q)(m \mod q) \mod q = (1^{j(p-1)} \mod q)(m \mod q) \mod q = (1 \mod q)(m \mod q) \mod q = 1(m \mod q) \mod q = (m \mod q) \mod q = m \mod q$. Pelo Teorema Chinês do Resto, como $m^{ed} \equiv m \pmod p$ e $m^{ed} \equiv m \pmod q$, podemos afirmar que $m^{ed} \equiv m \pmod{pq}$.

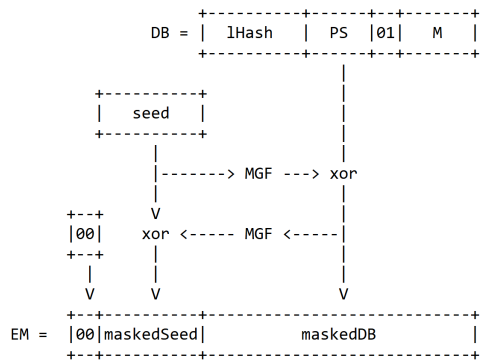

Figura 1. Esquema OAEP

3) Fixado o tamanho da mensagem, calculamos o *padding* PS, que consiste de $k - mLen - 2hLen - 2$ octetos de zeros (0x00).
4) Geramos o bloco de dados DB a partir da concatenação $lHash||PS||0x01||m$, sendo $0x01$ um octeto de separação entre o *padding* de zeros e a mensagem m. O bloco de dados DB terá tamanho fixo igual a $k - hLen - 1$.
5) Aleatoriamente, geramos a semente $seed$, do mesmo tamanho de $lHash$, isto é, $hLen$.
6) Sobre a semente $seed$, aplicamos uma função geradora de máscara (MGF), que produzirá uma máscara $dbMask$ do tamanho de DB, isto é, $k - hLen - 1$.
7) Realizamos uma operação XOR entre DB e $dbMask$ para obter o bloco de dados mascarado $maskedDB$.
8) Sobre o bloco de dados mascarado $maskedDB$, aplicamos a mesma função geradora de máscara anteriormente descrita (MGF), agora para produzir uma máscara $seedMask$ de tamanho $hLen$.
9) Realizamos uma operação XOR entre $seed$ e $seedMask$ para obter a semente mascarada $maskedSeed$.
10) Por fim, realizamos a seguinte concatenação: $0x00||maskedSeed||maskedDB$. O resultado dessa operação é a mensagem codificada EM. Sobre ela aplicaremos a operação de cifragem que caracteriza o sistema RSA.

Após a decifragem de uma mensagem pelo sistema RSA, para obter a mensagem original devemos reverter o processo de codificação acima descrito da seguinte maneira:

1) Decompomos a mensagem codificada em 0x00 (primeiro octeto), $maskedSeed$ ($hLen$ octetos seguintes) e $maskedDB$ (demais octetos).
2) Aplicamos a função geradora de máscara (MGF) sobre $maskedDB$ para obter $seedMask$.
3) Realizamos a operação XOR entre $maskedSeed$ e $seedMask$ para obter $seed$.
4) Aplicamos a função geradora de máscara (MGF) sobre $seed$ para obter $dbMask$.
5) Realizamos a operação XOR entre $maskedDB$ e $dbMask$ para obter DB.
6) Decompomos DB em $lHash$ (primeiros $hLen$ octetos), PS (octetos de zero seguintes), 0x01 (octeto de separa-

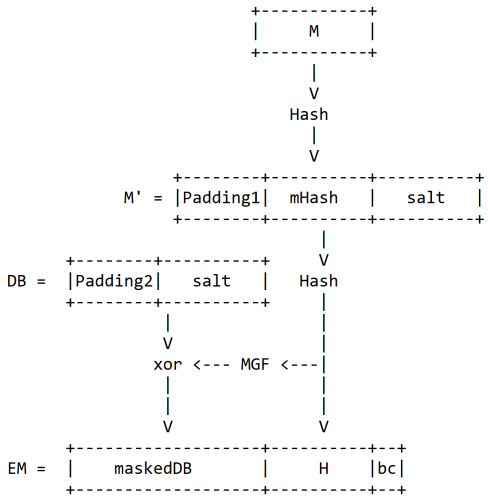

Figura 2. Esquema PSS

ção e, por fim, m.

Ao longo do processo de decodificação de EM, realizamos diversas checagens para garantir a integridade da mensagem: a) checamos se o primeiro octeto da mensagem codificada EM é igual a 0x00; b) verificamos se $lHash$ corresponde à aplicação da função de *hash* à $label$; e c) conferimos a existência do octeto de separação 0x01 em DB.

C. PSS

Similarmente ao processo de cifragem e decifragem, aplicamos ao processo de geração e verificação de assinatura digital um esquema (o PSS) para acrescentar um elemento de aleatoriedade, previamente à operação de cifragem pelo sistema RSA. A Figura 2, extraída de [2], sintetiza a lógica do esquema PSS.

A aplicação do esquema PSS se desdobra nos seguintes passos:

1) Sobre a mensagem m que desejamos assinar, aplicamos uma função de *hash* para obter a sequência $mHash$ com $hLen$ octetos (*bytes*).
2) Aleatoriamente, geramos a sequência $salt$ de tamanho de $sLen$.
3) Produzimos a sequência *Padding1* com oito octetos 0x00.
4) Realizamos a concatenação *Padding1*$||mHash||salt$ e a denominamos m'.
5) Sobre m' aplicamos uma função de *hash* para obter a sequência H, com o mesmo tamanho de $mHash$, isto é, $hLen$.
6) Estipulamos o tamanho útil $emBits$ (em *bits*) da assinatura codificada EM, observando que ele deve ser menor que o tamanho $modBits$ (em *bits*) do componente n das chaves, isto é, garantindo que $emBits < modBits$.
7) Calculamos a quantidade $emLen$ de octetos necessários para representar $emBits$ ($emLen = \lceil emBits/8 \rceil$).
8) Geramos a sequência PS com $emLen - sLen - hLen - 2$ octetos 0x00 e a concatenamos com o octeto separador 0x01, ao final, formando *Padding2*.
9) Compomos o bloco de dados DB por meio da concatenação *Padding2*$||salt$.
10) Aplicamos uma função geradora de máscara (MGF) sobre H para gerar uma máscara $dbMask$ do mesmo tamanho de DB, isto é, $emLen - hLen - 1$ octetos.
11) Para gerar a sequência $maskedDB$, realizamos uma operação XOR entre DB e $dbMask$ e, em seguida, zeramos os $8.emLen - emBits$ primeiros *bits* do resultado.
12) Produzimos a concatenação $maskedDB||H||0xbc$. Assim, obtemos a assinatura codificada EM, que será cifrada pelo sistema RSA.

Dadas uma mensagem m, em claro, e uma assinatura EM gerada a partir de m pelo esquema PSS, podemos verificar a integridade e a autenticidade da mensagem assinada da seguinte maneira:

1) Decompomos a assinatura codificada EM em três partes: 0xbc (o último octeto); H (os $hLen$ octetos antecedentes a 0xbc); e $maskedDB$ (os octectos iniciais restantes).
2) Aplicamos a função geradora de máscaras (MGF) sobre H para obter $dbMask$.
3) Realizamos uma operação XOR entre $maskedDB$ e $dbMask$ para obter DB.
4) Decompomos DB para obter $salt$ (os $sLen$ últimos octetos).
5) Aplicamos a função de *hash* sobre m para obter $mHash$.
6) Geramos o *Padding1* a partir da concatenação de oito octetos 0x00.
7) Realizamos a concatenação *Padding1*$||mHash||salt$ para obter m'.
8) Aplicamos a função de *hash* sobre m' e comparamos o resultado a H: se o resultado for igual, estão verificadas a integridade e a autenticidade da mensagem assinada.

Durante o processo de decodificação de EM, também realizamos uma série de checagens: a) observamos se o octeto final é igual a 0xbc; b) checamos se os $8.emLen - emBits$ primeiros *bits* de EM estão zerados; e c) verificamos se $Padding2$ é composto por octectos 0x00 no início e pelo octeto separador 0x01 ao final.

III. Ambiente experimental e análise de resultados

A. Descrição do cenário

Para ilustrar o funcionamento do sistema RSA, complementado pelos esquemas OAEP e PSS, implementamos um programa em linguagem Python (versão 3.13.13) com as funcionalidades de geração de chaves, cifragem e decifragem de mensagens e geração e verificação de assinaturas.

No código, a funcionalidade de geração de chaves é implementada pela função principal $gera_chaves$ e pelas funções auxiliares $gera_primo$, $gera_e$ e $aplica_miller_rebin$.

A função $gera_primo$ gera randomicamente valores ímpares no intervalo $[2^{1024}, 2^{1025})$ como candidatos a p e q, e

invoca a função $aplica_miller_rebin$ para testar, probabilisticamente, a primalidade desses valores, com base no algoritmo de Miller-Rebin.

A função $gera_e$ parte de um número primo inicial (*E_CANDIDATO*, parâmetro que fixamos em 65537) e testa sua coprimalidade com $\phi(n)$; em caso negativo, são considerados os números ímpares subsequentes até se encontrar um coprimo de $\phi(n)$.

As funcionalidades de cifragem e decifragem do sistema RSA são implementadas pela função $cifra_decifra$ que utiliza a função pow, nativa da linguagem de programação Python, para realizar as operações de exponenciação modular.

O esquema OAEP é implementado pelas funções $empacota_oaep$, que realiza o processo de codificação, e $desempacota_oaep$, responsável pelo processo de decodificação.

Similarmente, o esquema PSS é implementado pelas funções $empacota_pss$ e $desempacota_pss$. Nesse contexto, a função $assina_mensagem$ é responsável por disparar o processo de codificação realizado por $empacota_pss$.

A função de *hash* utilizada nos diversos momentos do processo de codificação e decodificação dos esquemas OAEP e PSS é a função *sha3_256*, que gera *hashes* de 32 *bytes* e foi implementada pela biblioteca $hashlib$.

Como função geradora de máscaras (MGF), implementamos a função $aplica_MGF1$, que concatena *hashes* gerados pela função *sha3_256* com contadores para produzir máscaras nos tamanhos desejados.

Por meio do terminal, o usuário escolhe a opção de ação que deseja executar (*1 - Gerar chaves públicas e privadas para A e B*, *2 - Cifrar mensagem de A para B*, *3 - Decifrar mensagem de A para B*, *4 - Assinar mensagem de A para B* ou *5 - Verificar assinatura de mensagem de A para B*).

Os resultados são sempre impressos no terminal, meio pelo qual também se recebem as chaves (pública ou privada) e as mensagens (a cifrar, a decifrar ou assinada), sempre codificadas em base64.

As chaves pública (n, e) e privada (n, d), assim como a mensagem assinada, são informadas e recebidas em formato JSON, funcionalidade implementada pelas funções $formata_chave$, $desformata_chave$ e $assina_mensagem$.

B. Análise de resultados

Para ilustrar o funcionamento do programa, selecionamos, inicialmente, a opção *1 - Gerar chaves públicas e privadas para A e B*. Após execução do programa, obtivemos a seguinte resposta, com as chaves geradas:

===== CHAVE PÚBLICA DE A =====
eyJuIjogIjdQSzFhdHg0QjlwRWVlc2RMem1U
Qno2UE1NVnNjaDJGUHhPaWZBUUNjNGR1V01O
ajBldkM5NWlxZGRLYUVIdGRycGdLTnhKTGtw
ay9hd0xRQTNMcFRKU1p5SWdpSjRUM1g0R0JO
YzRYOHNhTW1LTkxIbXVGY2wxT3RtTCtmaXBv
cjVBejRNeG5rVTNjYldEcDFzbEZXQWorNk5z
Q2dpUVU2aHZ3MXduRXJqdTVnYVBOcG5NcEtI
VFBBSXh3U2FyQzB3cG9RNDJIYXBscE5WbHRp
NVRiY3Q3WUwxa2RqUTNPWm5PTVZUa2htR1dI
T08rMTh3bTA1MVNQOVBOU21NRGlNTE9Ddy9y
MW5lYzF5R3BkNFZjMXNrK0lVT2huL1F4cmIw
Y2xCSkpmZXM2WmlWVWJjall1dFlNaUEvcWtM
MzVDWUNZQ2xIMnZpcm16WERQeUhuWG5pUT09
IiwgImUiOiAiQVFBQiJ9

===== CHAVE PRIVADA DE A =====
eyJuIjogIjdQSzFhdHg0QjlwRWVlc2RMem1UQ
no2UE1NVnNjaDJGUHhPaWZBUUNjNGR1V01Oaj
BldkM5NWlxZGRLYUVIdGRycGdLTnhKTGtway9
hd0xRQTNMcFRKU1p5SWdpSjRUM1g0R0JOYzRY
OHNhTW1LTkxIbXVGY2wxT3RtTCtmaXBvcjVBe
jRNeG5rVTNjYldEcDFzbEZXQWorNk5zQ2dpUV
U2aHZ3MXduRXJqdTVnYVBOcG5NcEtIVFBBSXh
3U2FyQzB3cG9RNDJIYXBscE5WbHRpNVRiY3Q3
WUwxa2RqUTNPWm5PTVZUa2htR1dIT08rMTh3b
TA1MVNQOVBOU21NRGlNTE9Ddy9yMW5lYzF5R3
BkNFZjMXNrK0lVT2huL1F4cmIwY2xCSkpmZXM
2WmlWVWJjall1dFlNaUEvcWtMMzVDWUNZQ2xI
MnZpcm16WERQeUhuWG5pUT09IiwgImQiOiAie
UpaTzdqL2xmUUJScDhSWjJIaXM5eDBHcXdyMl
cwU1NZR3VHeWVXN0RZSkE3L080dFNVWXovSDF
tL2pGWTk2TDJCbnR5K0U1Q01taTNqeHJUQUFq
Y0NqOFpyNlg2MHlzL2JUc1pWZ3EvZU1IY3RBS
jc1Nm1RY0tydDhybUZuaGdWditFYTNHZTIydE
doM0hiZzNDdnUyOGlISTRLK1pFQ0E0M1R2TGN
RUVUyOUFnK21QdlBWaUhBQ3FMTnptZm5CSTRw
MzJlK1dHZ0E5SmVqWnNpeGYvdm5WUEtKL3diW
mZiT0s2WDNML0xVem1XSmZINGFZYUNDK0tCVD
QxYWZNc1J1cWtJQkloM2RVWlM3TDNldFpIdzF
lK1VrcVhlaXNleTFsZUFVSEtuaUVNVVNSSzBu
NGQ4MG54M1JTZGcrQUNUa3VOTHFreFVSeWJtN
UFuOVlvS1lRPT0ifQ==

===== CHAVE PÚBLICA DE B =====
eyJuIjogImZlT1NaMkpnTVpVa2RDT3ZNYmxlR
Hk1Z3phN1pPblpCVmVmYjMrUmw5dWR0ZUR3ak
JkVnBrNzNJazBSMCtDY0M2cW8rcyswQWlVYWx
ObGM2VkUvRi9UeWFyRzNmT212Y2JXb0p6czBZ
cjRMcFBGL25YV0FJZFZ5dFdtVnNUdkVGcXNIM
HFBblZKUVhleTJzdiszSzF6YmIwdGswVFEraE
VYL3p6NVJmbHBmc0ErcWZ5YWR6U1djWlc2ZHY
zNWN4NUFvU1c3WkdWODJZMXRtUnRGVGZ0QUxY
TS84SnZ6djBQYjd0bWk0Rk5KWnhHNXNEdmJtS
3VsMDRGZ3pUQXdJYllHbjdBV2dEcnBsb0RVZm
5wZXVUWnJtTy8ybmxYa1V5T29YNkd6eU5hRUN
yVkNrSU5oNVM1SWliOXU1VVRReXpVdWkwTng4
ZUlwZjc4Q3hDamE3YXVPUT09IiwgImUiOiAiQ
VFBQiJ9

===== CHAVE PRIVADA DE B =====
eyJuIjogImZlT1NaMkpnTVpVa2RDT3ZNYmxlR
Hk1Z3phN1pPblpCVmVmYjMrUmw5dWR0ZUR3ak
JkVnBrNzNJazBSMCtDY0M2cW8rcyswQWlVYWx
ObGM2VkUvRi9UeWFyRzNmT212Y2JXb0p6czBZ
cjRMcFBGL25YV0FJZFZ5dFdtVnNUdkVGcXNIM
HFBblZKUVhleTJzdiszSzF6YmIwdGswVFEraE

VYL3p6NVJmbHBmc0ErcWZ5YWR6U1djWlc2ZHY
zNWN4NUFvU1c3WkdWODJZMXRtUnRGVGZ0QUxY
TS84SnZ6djBQYjd0bWk0Rk5KWnhHNXNEdmJtS
3VsMDRGZ3pUQXdJYllHbjdBV2dEcnBsb0RVZm
5wZXVUWnJtTy8ybmxYa1V5T29YNkd6eU5hRUN
yVkNrSU5oNVM1SWliOXU1VVRReXpVdWkwTng4
ZUlwZjc4Q3hDamE3YXVPUT09IiwgImQiOiAiT
UtPQmlGSTVDazh1WTNUNkZvTWhOVXBjK25zbz
BPNnB0YkE4Z3BlelVaNys2NnFJZVpPSUZCNEt
aZVBZTVp5WkF1TytFRFJzdFoxRzdlVHIwM0Z5
OUxHNnJRNVNaamNRVm5IVzBrRHAzYjM5c1diN
GRGcExTbFVCWVdDNFpSTzRkYzQwWWN4U3Z0Z3
l5WE5qZVlwTGRlelBSYlFuMGwvbU14UC9nNjh
rK21paVRJMDNvTTlvSGpaL2ZGSzN2MW9iSUNU
YWlkSDdyWFo3Unp2QTI0b3kvQXZtL3I2TnFET
3J6SUhGYXFHaFJ5cldkRGU3TXA1aVk3Y1JSY2
lTY204T2RPU2JuZEI4TFFCS3hvTnp0Q2RaMXl
FZk5UN3JNV3hsbU8xNG5ReXNtZldaSjdTR0h0
VDFEZEFoczRmZWtWaGdpSnorN1crSkx3RlcwU
k1seVkzaWxRPT0ifQ==

Geradas as chaves, acionamos a opção *2 - Cifrar mensagem de A para B* e informamos a mensagem *Esta é uma mensagem de teste!* e a chave pública de B (destinatário). Como resposta, a mensagem cifrada recebida foi a seguinte:

Ys3aK50XGg+dg9+ImKYaNjXCQXxziESPTxOo
HvQyULI147r42q7zqqbnpOupcIceqpalCOfO
T4hminQCC0SRrSu4u3j+Ci640ThBv0W6CpWi
7h6RzbsmHGSrXXJxZWNEutd4DhEA3dKPudHb
A2DUyZEPmVa546d4n44WjhMGJOdqsJFlL6Ke
y1I0H72nZH+Invacna2mCQRVyTb/gG0fRHHG
mdVcsKxeondPM4FAFoh3Da2ts/MYoVp00alI
RynCKtEbbcTcnM8IUKn6R3IetbeAlxJKptNU
YZseIfwPEgfCgoDczys5fsQVjAUBb20B406S
XkhOzenjdt6VDbMMHA==

Em seguida, selecionamos a opção *3 - Decifrar mensagem de A para B* e informamos a mensagem cifrada e a chave privada de B (destinatário). Como resultado, recebemos o texto: *Esta é uma mensagem de teste!*

Em sequência, escolhemos a opção *4 - Assinar mensagem de A para B* e informamos a mensagem *Esta é uma mensagem de teste!* e a chave privada de A (remetente). O resultado obtido foi a seguinte mensagem assinada:

eyJtZW5zYWdlbSI6ICJFc3RhIFx1MDBlOSB1b
WEgbWVuc2FnZW0gZGUgdGVzdGUhIiwgImFzc2
luYXR1cmEiOiAiZGk0U3V3V3QxK3FHc0o2dTh
HcWlUZUZMR1BubkRWdkJ4d1l5bHlHQVVzVG95
SWh5TGFsWndNdHBoWkoxay96YU4rc2R2Yllyb
XlNR3ZQZnhxTE1vamE5SkhHcTZpU2VKTFVNcE
toczZQbVBibi84S3BGNldSZU5Ub0ozc2dkeEp
QRnFpTW5VOXp6S1h2TjFtRElRZTFDSDhaSjBw
TlRIdm8reTJKaklaVWl3bUZTRDNEL1Fvdnhib
k1rWDd0c2w0ZUtjdlpwbTdKRHgxZThkWEttS1
l6bEMxaHorK1BNSGxyL3FOclJRd1hTY3lKUS8
yVGxRdlRqNEdqZjdMZ24zdzB0SWdreGUyS2VN
eEp0dHBJcnhsbmFBZWhFQVAxd1B4Zm1pOVl4e
VFDdmxYcUk1K2lhWGlSSFM5V3p0Szg5ZVQrcW
45SXVyVWo5SnhTQklYMW1CUERnPT0ifQ==

Por fim, selecionamos a opção *5 - Verificar assinatura de mensagem de A para B* e informamos a mensagem assinada recebida e a chave pública de A (remetente). Em resposta, obtivemos a confirmação de integridade e autenticidade da mensagem *Esta é uma mensagem de teste!*.

Realizamos as operações em máquina com as especificações descritas na Tabela I.

Tabela I
ESPECIFICAÇÕES DA MÁQUINA UTILIZADA PARA O EXPERIMENTO

Componente	**Descrição**
Frequência máxima de clock	2,1 GHz
Processador	12th Gen Intel(R) Core(TM) i7-1260P (Intel64 Family 6 Model 154 Stepping 3 GenuineIntel)
Número de núcleos	12
Número de processadores lógicos	16
Memória física total	33,94 GB
Sistema operacional	Microsoft Windows 11 Home Single Language x64-based PC (versão 10.0.26200)

A Tabela II indica o tempo médio de CPU consumido pelas principais operações executadas pelo programa.

Tabela II
TEMPO MÉDIO DE CPU

Operações	**Iterações**	**Tempo médio de CPU**
Geração de primo	100	3584,84 ms
Codificação OAEP	10000	0,00234 ms
Decodificação OAEP	10000	0,04531 ms
Cifragem RSA (mensagem)	10000	0,16406 ms
Cifragem RSA (assinatura)	100	33,125 ms
Decifragem RSA (mensagem)	100	21,71875 ms
Decifragem RSA (assinatura)	10000	0,19375 ms
Codificação PSS	10000	0,0375 ms
Decodificação PSS	10000	0,03906 ms

IV. CONCLUSÕES

O projeto desenvolvido demonstrou, em ambiente computacional convencional, a viabilidade técnica de implementar o sistema RSA (complementado pelos esquemas OAEP e PSS) como ferramenta de cifragem e de assinatura digital de mensagens.

Os testes evidenciaram que, enquanto as operações relativas aos esquemas OAEP e PSS são computacionalmente pouco custosas (ordem de 10^{-2} ms), o custo computacional de geração dos números primos que compõem as chaves do sistema RSA é significativo (ordem de 10^3 ms).

De outro, os testes mostraram que as operações de exponenciação modular que estão na base do sistema RSA têm custo computacional variável de acordo com a magnitude dos expoentes considerados.

No caso de teste, por exemplo, quando realizamos operações com o expoente e (ordem de 10^4), observamos tempo médio de CPU da ordem de 10^{-1} ms. Já quando operamos com o expoente d (ordem de 10^{230}), verificamos incremento no tempo médio de CPU da operação (ordem de 10^1 ms).

Adicionalmente, registramos a limitação do sistema RSA quanto ao tamanho da mensagem a ser cifrada, que deve ser menor que o valor de n. No caso de teste, em que n era da ordem de 2^{2048}, o teto de tamanho da mensagem era de 256 *bytes* ou 256 caracteres ASCII (sem considerar que os esquemas OAEP e PSS consomem muitos desses *bytes*). Esse fato evidencia as limitações em utilizar o sistema RSA como mecanismo de cifragem de mensagens extensas.

Referências

[1] Contribuidores da Wikipédia, *RSA cryptosystem*, Wikipedia, The Free Encyclopedia [Online]. 31 mar. 2026, 17h32min. UTC. Disponível: https://en.wikipedia.org/w/index.php?title=RSA_cryptosystem&oldid=1346408321.

[2] B. S. Kaliski (Ed.), *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, Internet Engineering Task Force, 2016. [Online]. Disponível: https://doi.org/10.17487/RFC8017.

[3] Contribuidores da Wikipédia, *Euler's totient function*, Wikipedia, The Free Encyclopedia [Online]. 28 mar. 2026, 13h33min. UTC. Disponível:https://en.wikipedia.org/w/index.php?title=Euler%27s_totient_function&oldid=1345837814.

Apêndice
Código do programa

A íntegra do código do programa desenvolvido pode ser consultada abaixo ou no repositório do projeto no GitHub: https://github.com/andmilhomem/rsa.

```python

import secrets
import math
import base64
import hashlib
import json
import random
import time # Para testes

# Parâmetros:
TAM_HASH_BYTES = 32 # SHA-256
LABEL_HASH = b"" # Usada para gerar hash de incio do pacote OAEP
TAM_P_Q_BITS = 1024 # O teto para n (e, portanto, para m)  o dobro desse valor
E_CANDIDATO = 65537
TESTE = False # Alterar para ativar testes
ITERACOES_TESTE_PRIMO = 100
ITERACOES_TESTE_E = 10000
ITERACOES_TESTE_D = 100
ITERACOES_TESTE_ENC_DEC = 10000

# Funções auxiliares:
def int_para_base64(i):
    return base64.b64encode(i.to_bytes((i.bit_length() + 7) // 8, "big"))

def base64_para_int(b):
    return int.from_bytes(base64.b64decode(b),"big")

def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def aplica_miller_rebin(n):
    k=40
    # casos básicos
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    # escreve n-1 = d * 2^r
    d = n - 1
    r = 0
    while d % 2 == 0:
        d //= 2
        r += 1

    # testes
    for _ in range(k):
        a = random.randrange(2, n - 2)
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
```

```python
            return False

    return True

def aplica_mgf1(semente, tam_mascara):
    resultado = b""
    for cont in range(math.ceil(tam_mascara / TAM_HASH_BYTES)):
        cont_bytes = cont.to_bytes(4,"big")
        resultado += hashlib.sha3_256(semente + cont_bytes).digest()
    return resultado[:tam_mascara]

def gera_primo(qtde_bits):
    min_val = 1 << (qtde_bits - 1)
    max_val = (1 << qtde_bits ) - 1

    e_primo = False
    while not e_primo:
        candidato = secrets.randbelow(max_val - min_val + 1) + min_val # gera candidato no intervalo desejado
        candidato = candidato | 1 # transforma em mpar, se necessrio
        e_primo = aplica_miller_rebin(candidato) # checa primalidade

    return candidato

def gera_e(phi_n):
    e = E_CANDIDATO
    while not math.gcd(e, phi_n) == 1:
        e += 2
    return e

def formata_chave(n, e_ou_d, tipo):
    # Compõe JSON
    chave_formatada = ""
    if (tipo == "publ"):
        chave_formatada = json.dumps({
            "n": int_para_base64(n).decode('utf-8'),
            "e": int_para_base64(e_ou_d).decode('utf-8')
        })
    elif (tipo == "priv"):
        chave_formatada = json.dumps({
            "n": int_para_base64(n).decode('utf-8'),
            "d": int_para_base64(e_ou_d).decode('utf-8')
        })

    # Converte JSON para base64
    chave_formatada_base64 = base64.b64encode(chave_formatada.encode("utf-8"))

    return chave_formatada_base64.decode("utf-8")

def desformata_chave(chave):
    # Decompõe JSON
    chave_json = json.loads(base64.b64decode(chave).decode("utf-8"))
    n_base64 = chave_json.get("n")
    d_base64 = chave_json.get("d")
    e_base64 = chave_json.get("e")

    if d_base64 != None:
        return base64_para_int(n_base64), base64_para_int(d_base64)
    else:
        return base64_para_int(n_base64), base64_para_int(e_base64)

# Funções principais:
def gera_chaves():
    if TESTE == True:
        inicio = time.process_time_ns()
        for i in range(ITERACOES_TESTE_PRIMO):
            p = gera_primo(TAM_P_Q_BITS)
        fim = time.process_time_ns()
        print(f"\nTempo de {ITERACOES_TESTE_PRIMO} geraes de primos ({fim-inicio} ns):")

    p = gera_primo(TAM_P_Q_BITS)
    q = gera_primo(TAM_P_Q_BITS)
    n = p*q
    phi_n = (p-1)*(q-1)
    e = gera_e(phi_n)
    d = pow(e, -1, phi_n) # pow(a,b,c) = a^b mod c, ou seja, d = e^(-1) mod phi_n o inverso modular de e
    return n, e, d

def cifra_decifra(base, expoente, n):
    return pow(base, expoente, n)

def empacota_oaep(m, n):
    # Calcula tamanho mximo da mensagem a partir de 'n'
    tam_n_em_bytes = (n.bit_length() + 7) // 8
    tam_max_m_em_bytes = tam_n_em_bytes - 2 - (2 * TAM_HASH_BYTES) # -1 (0x00 na EM porque m < n), -1 (0x01 no DB como delimitador), -TAM_HASH_BYTES (hash do DB), -TAM_HASH_BYTES (masked_seed de EM)

    # Checa tamanho de 'm'
    tam_m = len(m)
    if tam_m > tam_max_m_em_bytes:
        return -1, f"\nO tamanho da mensagem ({tam_m} bytes) excedeu o limite mximo ({tam_max_m_em_bytes} bytes)!"

    # Gera padding de zeros
    tam_padding = tam_max_m_em_bytes - tam_m
    padding = b"\x00" * tam_padding

    # Gera hash do pacote
    h = hashlib.sha3_256(LABEL_HASH).digest()

    # Compõe pacote (gera data block)
    pacote = h + padding + b"\x01" + m

    # Mascara pacote (gera masked data block)
    semente = secrets.token_bytes(TAM_HASH_BYTES)
    mascara_pacote = aplica_mgf1(semente, len(pacote))
    pacote_mascarado = xor_bytes(pacote, mascara_pacote)

    # Mascara semente (gera masked seed)
```

```python
    mascara_semente = aplica_mgf1(pacote_mascarado, TAM_HASH_BYTES)
    semente_mascarada = xor_bytes(semente, mascara_semente)

    # Compe pacote final (gera encoded message) e converte para int
    pacote_final = int.from_bytes(b"\x00" + semente_mascarada + pacote_mascarado,"big")
    return pacote_final, ""

def desempacota_oaep(pacote):
    # Decompõe pacote
    if pacote[0] != 0x00:
        return -1, "início diferente de 0x00"
    semente_mascarada = pacote[1: 1 + TAM_HASH_BYTES]
    pacote_mascarado = pacote[1 + TAM_HASH_BYTES:]

    # Desmascara semente
    mascara_semente = aplica_mgf1(pacote_mascarado, TAM_HASH_BYTES)
    semente_desmascarada = xor_bytes(semente_mascarada, mascara_semente)

    # Desmascara pacote
    mascara_pacote = aplica_mgf1(semente_desmascarada, len(pacote_mascarado))
    pacote_desmascarado = xor_bytes(pacote_mascarado, mascara_pacote)

    # Desempacota mensagem
    h = pacote_desmascarado[0:TAM_HASH_BYTES]
    resto = pacote_desmascarado[TAM_HASH_BYTES:]
    if h != hashlib.sha3_256(LABEL_HASH).digest(): # Checa hash
        return -1, "hash inválido"
    indice_separador = resto.find(b'\x01') # Checa separador
    if indice_separador == -1:
        return -1, "separador não localizado"
    padding = resto[:indice_separador]
    m_desempacotada = resto[indice_separador + 1:]
    for b in padding: # Checa padding
        if b != 0x00:
            return -1, "padding inválido"
    return 0, m_desempacotada.decode("utf-8")

def empacota_pss(m, n):
    # Calcula tamanho mximo da assinatura (encoded message) a partir de 'n'
    tam_max_a_bits = n.bit_length() - 1
    tam_max_a_bytes = (tam_max_a_bits + 7) // 8

    # Checa se tamanho mximo da assinatura (encoded message) comporta os elementos obrigatrios (0x01 + salt no db; H + 0xbc na em)
    if tam_max_a_bytes < 2*TAM_HASH_BYTES + 2:
        return -1

    # Gera m_linha
    padding1 = b"\x00" * 8
    h = hashlib.sha3_256(m).digest() #mHash
    salt = secrets.token_bytes(TAM_HASH_BYTES)
    m_linha = padding1 + h + salt

    # Gera padding de zeros do pacote
    tam_padding_pacote_bytes = tam_max_a_bytes - ( 2 * TAM_HASH_BYTES) - 2  # em menos (H + 0xbc) menos (0x01 + salt)
    padding_pacote = b"\x00" * tam_padding_pacote_bytes

    # Gera pacote (data block)
    pacote = padding_pacote + b"\x01" + salt

    # Gera hash final (H)
    h_final = hashlib.sha3_256(m_linha).digest()

    # Mascara pacote (gera masked data block)
    mascara_pacote = aplica_mgf1(h_final, len(pacote))
    pacote_mascarado = xor_bytes(pacote, mascara_pacote)

    # Zera os primeiros bits para garantir que 'em' no exceda 'n'
    bits_sobressalentes = (8 * tam_max_a_bytes) - tam_max_a_bits
    if bits_sobressalentes > 0:
        mascara =  (0xFF >> bits_sobressalentes)
        ba = bytearray(pacote_mascarado)
        ba[0] &= mascara
        pacote_mascarado_ajustado = bytes(ba)
    else:
        pacote_mascarado_ajustado = pacote_mascarado

    # Compe pacote final (gera encoded message) e converte para int
    pacote_final = int.from_bytes(pacote_mascarado_ajustado + h_final + b"\xbc","big")

    return pacote_final

def desempacota_pss(pacote, m, n):
    # Calcula tamanho mximo da assinatura (encoded message) a partir de 'n'
    tam_max_a_bits = n.bit_length() - 1
    tam_max_a_bytes = (tam_max_a_bits + 7) // 8

    # Checa se os bits sobressalentes esto zerados
    bits_sobressalentes = (8 * tam_max_a_bytes) - tam_max_a_bits
    if bits_sobressalentes > 0:
        mascara = 0xFF << (8 - bits_sobressalentes) & 0xFF
        if (pacote[0] & mascara) != 0:
            return -1, "bits sobressalentes no zerados"

    # Decompõe pacote
    if pacote[-1] != 0xbc:
```

```python
        return -1, "fim diferente de 0xbc"
    pos_h_final = len(pacote) - TAM_HASH_BYTES - 1
    pacote_mascarado = pacote[:pos_h_final]
    h_final_recebido = pacote[pos_h_final: pos_h_final + TAM_HASH_BYTES]

    # Desmascara pacote
    h = hashlib.sha3_256(m).digest()
    mascara_pacote = aplica_mgf1(h_final_recebido, len(pacote) - TAM_HASH_BYTES - 1)
    pacote_desmascarado = xor_bytes(pacote_mascarado, mascara_pacote)

    # Zera bits sobressalentes
    if bits_sobressalentes > 0:
        mascara = (0xFF >> bits_sobressalentes)
        ba = bytearray(pacote_desmascarado)
        ba[0] &= mascara
        pacote_desmascarado_ajustado = bytes(ba)
    else:
        pacote_desmascarado_ajustado = pacote_mascarado

    # Checa se padding está zerado
    tam_padding_pacote_bytes = tam_max_a_bytes - ( 2 * TAM_HASH_BYTES) - 2  # em menos (H + 0xbc) menos (0x01 + salt)
    padding = pacote_desmascarado_ajustado[:tam_padding_pacote_bytes]
    for b in padding: # Checa padding
        if b != 0x00:
            return -1, "padding inválido"

    # Checa se separador está correto
    if pacote_desmascarado_ajustado[tam_padding_pacote_bytes] != 1:
        return -1, "separador não localizado"

    # Calcula h_final
    salt = pacote_desmascarado_ajustado[tam_padding_pacote_bytes+1:]
    h = hashlib.sha3_256(m).digest() #mHash
    m_linha = b"\x00" * 8 + h + salt
    h_final = hashlib.sha3_256(m_linha).digest()

    # Verifica assinatura
    return 0, h_final_recebido == h_final

def assina_mensagem(m, n, d):
    # Gera e empacota assinatura
    a = empacota_pss(m, n)
    if a == -1: return a, "Erro ao assinar!"
    if TESTE == True:
        inicio = time.process_time_ns()
        for i in range(ITERACOES_TESTE_ENC_DEC):
            empacota_pss(m, n)
        fim = time.process_time_ns()
        print(f"\nTempo de {ITERACOES_TESTE_ENC_DEC} codificaes PSS ({fim-inicio} ns):")

    # Cifra assinatura
    a_cifrada = cifra_decifra(a, d, n)
    if TESTE == True:
        inicio = time.process_time_ns()
        for i in range(ITERACOES_TESTE_D):
            cifra_decifra(a, d, n)
        fim = time.process_time_ns()
        print(f"\nTempo de {ITERACOES_TESTE_D} cifragens RSA ({fim-inicio} ns):")
    a_cifrada_base64 = int_para_base64(a_cifrada)

    # Compõe mensagem e assinatura
    m_assinada = json.dumps({
        "mensagem": m.decode("utf-8"),
        "assinatura": a_cifrada_base64.decode('utf-8')
    })

    # Converte mensagem para base64
    m_assinada_base64 = base64.b64encode(m_assinada.encode("utf-8"))

    return 0, m_assinada_base64.decode("utf-8")


while True:
    print("\n==================== OPES ====================\n")
    print("1 - Gerar chaves pblicas e privadas para A e B")
    print("2 - Cifrar mensagem de A para B")
    print("3 - Decifrar mensagem de A para B")
    print("4 - Assinar mensagem de A para B")
    print("5 - Verificar assinatura de mensagem de A para B")
    print("\n================================================\n")

    try:
        match input("\n> "):
            case "1": # Gera chaves
                n_A, e_A, d_A = gera_chaves()
                n_B, e_B, d_B = gera_chaves()

                try:
                    chave_pblica_A = formata_chave(n_A, e_A, "publ")
                    chave_privada_A = formata_chave(n_A, d_A, "priv")
                    chave_pblica_B = formata_chave(n_B, e_B, "publ")
                    chave_privada_B = formata_chave(n_B, d_B, "priv")

                    print("\n============ CHAVE PBLICA DE A ===============\n")
                    print(chave_pública_A)
                    print("\n============ CHAVE PRIVADA DE A ===============\n")
                    print(chave_privada_A)
```

```python
                print("\n============ CHAVE PBLICA DE B ==============\n")
                print(chave_pública_B)
                print("\n============ CHAVE PRIVADA DE B ==============\n")
                print(chave_privada_B)
                print("\n================================================\n")
            except:
                print("\nErro ao gerar chaves!")

        case "2": # Cifra mensagem
            # Obtém 'chave'
            print("\nInforme a chave pblica de B:")
            try:
                n_B, e_B = desformata_chave(input("> "))
            except:
                print("\nErro ao processar chave!")
                continue

            # Obtém 'm'
            print(f"\nInforme a mensagem:")
            m = input("> ").encode("utf-8")

            # Empacota 'm'
            pacote, informacao = empacota_oaep(m, n_B)
            if TESTE == True:
                inicio = time.process_time_ns()
                for i in range(ITERACOES_TESTE_ENC_DEC):
                    empacota_oaep(m, n_B)
                fim = time.process_time_ns()
                print(f"\nTempo de {ITERACOES_TESTE_ENC_DEC} codificaes OAEP ({fim-inicio} ns):")
            if pacote == -1:
                print(informacao)
                continue

            # Cifra pacote (RSA)
            c = cifra_decifra(pacote, e_B, n_B)
            print(f"\nMensagem cifrada com sucesso:\n{int_para_base64(c).decode('utf-8')}")
            if TESTE == True:
                inicio = time.process_time_ns()
                for i in range(ITERACOES_TESTE_E):
                    cifra_decifra(pacote, e_B, n_B)
                fim = time.process_time_ns()
                print(f"\nTempo de {ITERACOES_TESTE_E} cifragens RSA ({fim-inicio} ns):")

        case "3": # Decifra mensagem
            # Obtém 'chave'
            print("\nInforme a chave privada de B:")
            try:
                n_B, d_B = desformata_chave(input("> "))
            except:
                print("\nErro ao processar chave!")
                continue

            # Obtém mensagem cifrada
            print(f"\nInforme a mensagem cifrada (em base64):")
            c = base64_para_int(input("> "))

            # Decifra mensagem
            pacote_decifrado_int = cifra_decifra(c, d_B, n_B)
            if TESTE == True:
                inicio = time.process_time_ns()
                for i in range(ITERACOES_TESTE_D):
                    cifra_decifra(c, d_B, n_B)
                fim = time.process_time_ns()
                print(f"\nTempo de {ITERACOES_TESTE_D} decifragens RSA ({fim-inicio} ns):")
            tam_n_em_bytes = (n_B.bit_length() + 7) // 8
            pacote_decifrado_bytes = pacote_decifrado_int.to_bytes(tam_n_em_bytes,"big")
            resultado, informacao = desempacota_oaep(pacote_decifrado_bytes)
            if TESTE == True:
                inicio = time.process_time_ns()
                for i in range(ITERACOES_TESTE_ENC_DEC):
                    desempacota_oaep(pacote_decifrado_bytes)
                fim = time.process_time_ns()
                print(f"\nTempo de {ITERACOES_TESTE_ENC_DEC} decodificaes OAEP ({fim-inicio} ns):")

            if resultado == -1:
                print(f"\nMensagem corrompida ({informacao})!")
                continue
```

```
                print(f"\nMensagem decifrada com sucesso:\n{informacao}")

            case "4": # Assina mensagem
                # Obtém 'chave'
                print("\nInforme a chave privada de A:")
                try:
                    n_A, d_A = desformata_chave(input("> "))
                except:
                    print("\nErro ao processar chave!")
                    continue

                # Obtém 'm'
                print(f"\nInforme a mensagem:")
                m = input("> ").encode("utf-8")

                # Assina mensagem
                resultado, informacao = assina_mensagem(m, n_A, d_A)
                if resultado == -1: print(informacao)
                else: print(f"\nMensagem assinada com sucesso:\n{informacao}")

            case "5": # Verifica assinatura
                # Obtém 'chave'
                print("\nInforme a chave pblica de A:")
                try:
                    n_A, e_A = desformata_chave(input("> "))
                except:
                    print("\nErro ao processar chave!")
                    continue

                # Obtém e decompõe a mensagem
                print(f"\nInforme a mensagem assinada (em base 64):")
                m_recebida = base64.b64decode(input("> ")).decode("utf-8")
                m_recebida_json = json.loads(m_recebida)
                m_recebida_bytes = m_recebida_json["mensagem"].encode("utf-8")
                a_recebida_base64 = m_recebida_json["assinatura"]

                # Decifra assinatura recebida
                a_recebida_int = base64_para_int(a_recebida_base64)
                a_decifrada_int = cifra_decifra(a_recebida_int, e_A, n_A)
                if TESTE == True:
                    inicio = time.process_time_ns()
                    for i in range(ITERACOES_TESTE_E):
                        cifra_decifra(a_recebida_int, e_A, n_A)
                    fim = time.process_time_ns()
                    print(f"\nTempo de {ITERACOES_TESTE_E} decifragens RSA ({fim-inicio} ns):")

                # Desempacota assinatura recebida
                tam_n_em_bytes = (n_A.bit_length() + 7) // 8
                a_decifrada_bytes = a_decifrada_int.to_bytes(tam_n_em_bytes,"big")
                resultado, informacao = desempacota_pss(a_decifrada_bytes,
                m_recebida_bytes, n_A)
                if TESTE == True:
                    inicio = time.process_time_ns()
                    for i in range(ITERACOES_TESTE_ENC_DEC):
                        desempacota_pss(a_decifrada_bytes, m_recebida_bytes, n_A)
                    fim = time.process_time_ns()
                    print(f"\nTempo de {ITERACOES_TESTE_ENC_DEC} decodificaes PSS ({fim-inicio} ns):")

                if resultado == -1:
                    print(f"\nAssinatura corrompida ({informacao})!")
                    continue
                if informacao == True:
                    print("\nIntegridade confirmada da seguinte mensagem:")
                    print(m_recebida_bytes.decode("utf-8"))
                else:
                    print("\nAssinatura invlida!")

            case _:
                print("\n Opo invlida. Tente novamente.\n")
except:
    exit()
```